

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: شایان عباسی

شماره دانشجویی: 04121040768022

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

پروژه 2: شبیه‌ساز چراغ راهنمایی هوشمند

شرح

یک چهارراه با چهار مسیر را شبیه‌سازی کنید.

امکانات

- ورود خودرو
- صف خودروها
- کنترل زمان چراغ‌ها
- نمایش گرافیکی

کلاس‌ها

Car

TrafficLight

Lane

Intersection

Controller

قابلیت توسعه

الگوریتم زمان‌بندی تطبیقی

1. کتابخانه ها

```
import tkinter as tk
```

در این خط، کتابخانه `tkinter` را وارد کردم و نام مستعار `tk` به آن دادم. از این کتابخانه برای ساخت پنجره اصلی برنامه و تمام عناصر گرافیکی مانند `Canvas`، دکمه‌ها و برجسب‌ها استفاده کردم.

```
from tkinter import ttk
```

از این خط برای وارد کردن ماژول `ttk` از `tkinter` استفاده کردم. `ttk` امکانات پیشرفته‌تری مانند `Label` و `Entry` را فراهم می‌کند که ظاهر بهتری نسبت به امکانات پیش‌فرض دارد.

```
import random
```

کتابخانه `random` را وارد کردم تا بتوانم به صورت تصادفی خودروها در مسیرهای مختلف ایجاد کنم. از این کتابخانه در تابع `new_car` استفاده کردم تا جهت ورود خودرو به چهارراه تصادفی انتخاب شود.

۲. کلاس Car (مدل خودرو)

```
class Car:
    def __init__(self, direction):
        self.direction = direction
        self.speed = 3
        if direction == "N":
            self.x = 335
            self.y = -20
        elif direction == "S":
            self.x = 365
            self.y = HEIGHT + 20
        elif direction == "E":
            self.x = WIDTH + 20
            self.y = 335
        else:
```

```
self.x = -20
self.y = 365
```

این کلاس را برای مدل‌سازی هر خودرویی که وارد چهارراه می‌شود طراحی کردم. در متد `__init__` جهت حرکت دریافت می‌شود و بر اساس آن موقعیت شروع خودرو در لبه تصویر تنظیم می‌شود. سرعت ثابت ۳ پیکسل در هر فریم است. خودروهای شمالی از بالا، جنوبی از پایین، شرقی از راست و غربی از چپ وارد می‌شوند.

۳. کلاس TrafficLight (مدیریت چراغ)

```
class TrafficLight:
    def __init__(self, direction):
        self.direction = direction
        self.mode = "RED"
```

این کلاس را برای نمایش هر چراغ راهنمایی در چهارراه ساختم. هر چراغ یک جهت دارد و حالت آن که RED، GREEN یا YELLOW است نگهداری می‌شود.

متد `set_mode` تغییر حالت چراغ

```
def set_mode(self, mode):
    self.mode = mode
```

از این متد برای تغییر حالت چراغ استفاده می‌شود. `Controller` هر بار که نوبت یک جهت می‌رسد این متد را صدا می‌زند تا چراغ مربوطه سبز شود و بقیه قرمز.

متد `get_color` برگرداندن رنگ

```
def get_color(self):
    if self.mode == "GREEN":
        return "green"
    elif self.mode == "YELLOW":
        return "yellow"
    else:
        return "red"
```

این متد رنگ مناسب برای رسم روی Canvas را برمی‌گرداند. این جداسازی کمک می‌کند منطق کنترل از رسم گرافیکی جدا بماند و کد خواناتر باشد.

۴. کلاس Lane (مدیریت صف خودروها)

```
class Lane:
    def __init__(self, direction):
        self.direction = direction
        self.queue = []
```

این کلاس را برای نمایش هر مسیر ورودی چهارراه ایجاد کردم. هر Lane یک جهت دارد و یک لیست خالی که خودروهای در صف را نگه می‌دارد.

متد add_car اضافه کردن خودرو

```
def add_car(self, car):
    self.queue.append(car)
```

از این متد برای اضافه کردن خودرو جدید به صف استفاده کردم. هر بار که new_car فراخوانی می‌شود یک خودرو ساخته شده و به Lane مربوطه اضافه می‌شود.

متد remove_car حذف خودرو

```
def remove_car(self, car):
    if car in self.queue:
        self.queue.remove(car)
```

این متد خودرویی که از صحنه خارج شده را حذف می‌کند. قبل از حذف بررسی می‌کند که آیا در لیست هست تا از خطای احتمالی جلوگیری شود.

متد count تعداد خودروها

```
def count(self):  
    return len(self.queue)
```

این متد تعداد خودروهای در صف را برمی‌گرداند که برای نمایش اطلاعات روی Canvas و الگوریتم تطبیقی استفاده می‌شود.

۵. کلاس Intersection (تقاطع)

```
class Intersection:  
    def __init__(self):  
        self.lanes = {"N": Lane("N"), "S": Lane("S"),  
                      "E": Lane("E"), "W": Lane("W")}  
        self.lights = {"N": TrafficLight("N"), ...}  
        self.positions = {"N": (350,260), "S": (350,440),  
                          "E": (440,350), "W": (260,350)}  
        self.stop_line = {"N": 290, "S": 410, "E": 410, "W": 290}
```

این کلاس همه اجزای چهارراه را کنار هم قرار می‌دهد. چهار Lane برای چهار مسیر، چهار TrafficLight برای چهار چراغ، موقعیت هر چراغ روی Canvas، و خط توقف هر مسیر که خودروها قبل از رسیدن به آن باید منتظر بمانند.

متد get_queue_counts تعداد خودروهای هر مسیر

```
def get_queue_counts(self):  
    return {d: self.lanes[d].count() for d in self.lanes}
```

این متد تعداد خودروهای منتظر در هر مسیر را به صورت یک دیکشنری برمی‌گرداند. این اطلاعات توسط Controller برای الگوریتم تطبیقی استفاده می‌شود تا مسیر شلوغ‌تر زمان بیشتری بگیرد.

۶. کلاس Controller (مغز کنترل چراغ‌ها)

```
class Controller:

    def __init__(self, intersection, green_time, yellow_time):

        self.intersection = intersection

        self.dirs = ["N", "S", "E", "W"]

        self.current_index = 0

        self.active = self.dirs[0]

        self.mode = "GREEN"

        self.timer = self.green_time
```

این کلاس را به عنوان مغز متفکر سیستم طراحی کردم. ارجاع به تقاطع و زمان‌های سبز و زرد را دریافت می‌کند. ترتیب چرخش چراغ‌ها در لیست `dirs` مشخص است و `timer` برای شمارش معکوس استفاده می‌شود.

متد tick یک قدم زمانی

```
def tick(self, dt=0.05):

    self.timer -= dt

    if self.timer <= 0:

        if self.mode == "GREEN":

            self.mode = "YELLOW"

            self.timer = self.yellow_time

        else:

            self.current_index = (self.current_index + 1) % 4

            self.active = self.dirs[self.current_index]

            self.mode = "GREEN"

            self.timer = self.green_time

    self._apply_lights()
```

این متد بخش منطق چرخش چراغ‌هاست و هر ۵۰ میلی‌ثانیه یک بار فراخوانی می‌شود. تایمر را کاهش می‌دهد و وقتی به صفر رسید ابتدا به حالت `YELLOW` می‌رود و بعد به چراغ بعدی می‌چرخد. چرخش با `modulo 4` انجام می‌شود تا پس از `W` دوباره به `N` برگردد.

متد adaptive_tick زمان‌بندی تطبیقی

```
def adaptive_tick(self, dt=0.05):
    counts = self.intersection.get_queue_counts()
    if self.mode == "GREEN":
        extra = min(counts[self.active] * 0.5, 5)
        if self.timer + extra <= 0:
            self.mode = "YELLOW"
            self.timer = self.yellow_time
    else:
        busiest = max(self.dirs, key=lambda d: counts[d])
        if busiest != self.active and counts[busiest] > counts[self.active]+2:
            self.current_index = self.dirs.index(busiest)
        else:
            self.current_index = (self.current_index + 1) % 4
```

این متد قابلیت توسعه الگوریتم تطبیقی را پیاده‌سازی می‌کند. اگر صف خودروهای مسیر فعلی شلوغ باشد، زمان سبز تا ۵ ثانیه اضافه می‌شود. بعد از اتمام، اگر مسیر دیگری بیش از ۲ خودرو بیشتر داشته باشد، نوبت مستقیم به آن مسیر می‌رود تا ترافیک متعادل‌تر شود.

۷. کلاس TrafficApp (رابط گرافیکی)

```
class TrafficApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Smart Traffic Light")
        self.canvas = tk.Canvas(root, width=WIDTH, height=HEIGHT, bg="white")
        self.canvas.pack()
        self.green = tk.IntVar(value=6)
        self.yellow = tk.IntVar(value=2)
        self.adaptive_var = tk.BooleanVar(value=False)
```

این کلاس را برای مدیریت کامل رابط گرافیکی طراحی کردم. Canvas برای رسم چهارراه، Entry برای تنظیم زمان چراغ‌ها، و Checkbutton برای فعال کردن حالت تطبیقی. متغیرهای IntVar و BooleanVar به طور خودکار با عناصر UI همگام می‌مانند.

متد new_car ورود خودرو تصادفی

```
def new_car(self):
    d = random.choice(["N", "S", "E", "W"])
    car = Car(d)
    self.intersection.lanes[d].add_car(car)
```

این متد با احتمال ۵٪ در هر فریم یک خودرو تصادفی می‌سازد. جهت ورود به صورت تصادفی از چهار جهت N، S، E، W انتخاب می‌شود و خودرو به Lane مربوطه اضافه می‌شود.

متد move_cars حرکت خودروها

```
def move_cars(self):
    stop = self.intersection.stop_line
    for d in self.intersection.lanes:
        light_mode = self.intersection.lights[d].mode
        for car in lane.queue:
            go = (light_mode == "GREEN")
            if go or car.y < stop[d]: # مثال برای جهت N
                car.y += car.speed
```

این متد برای هر خودرو بررسی می‌کند آیا چراغش سبز است یا هنوز به خط توقف نرسیده. اگر چراغ سبز باشد خودرو ادامه می‌دهد. اگر قرمز باشد و به خط توقف رسیده باشد، متوقف می‌ماند تا چراغ سبز شود. خودروهایی که از تصویر خارج شوند حذف می‌شوند.

متد draw رسم تصویر چهارراه

```
def draw(self):
    c = self.canvas
    c.delete("all")
    c.create_rectangle(300, 0, 400, HEIGHT, fill="gray")
    c.create_rectangle(0, 300, WIDTH, 400, fill="gray")
    for d, light in self.intersection.lights.items():
        x, y = self.intersection.positions[d]
        c.create_oval(x-8, y-8, x+8, y+8, fill=light.get_color())
    for car in self.intersection.lanes[d].queue:
        c.create_rectangle(car.x-6, car.y-10, car.x+6, car.y+10, fill="blue")
```

این متد هر بار که **update** اجرا می‌شود کل بوم را پاک کرده و دوباره رسم می‌کند. دو مستطیل خاکستری جاده‌های عمودی و افقی را می‌سازند. چراغ‌ها به صورت دایره با رنگ واقعی رسم می‌شوند. هر خودرو یک مستطیل آبی کوچک است. در گوشه چپ بالا تعداد خودروهای هر مسیر نمایش داده می‌شود.

متد **update** حلقه اصلی برنامه

```
def update(self):
    if random.random() < 0.05:
        self.new_car()
    self.controller.green_time = self.green.get()
    self.controller.yellow_time = self.yellow.get()
    if self.adaptive_var.get():
        self.controller.adaptive_tick()
    else:
        self.controller.tick()
    self.move_cars()
    self.draw()
    self.root.after(50, self.update)
```

در هر اجرا ابتدا شانس ۵٪ برای ورود خودرو بررسی می‌شود، سپس زمان‌های چراغ از **Entry** خوانده می‌شوند تا تغییرات کاربر بلافاصله اعمال شوند. بر اساس وضعیت **Checkbox**، حالت عادی یا تطبیقی انتخاب می‌شود. از **root.after(50)** استفاده کردم تا بعد از ۵۰ میلی‌ثانیه خودش را فراخوانی کند.

۸. نقطه ورودی برنامه

```
root = tk.Tk()
app = TrafficApp(root)
root.mainloop()
```

این بلوک نقطه شروع برنامه است. یک پنجره **tkinter** ساخته می‌شود، آن را به **TrafficApp** می‌دهد و با **mainloop()** برنامه را در حالت اجرا نگه می‌دارد تا کاربر پنجره را ببندد.